# From 16 MB to 110 KB: Shrinking MobileNetV3 for Satellite Image Classification

*Structured pruning and int8 quantization on EuroSATVersion: 08.07.2026*

*Authors: Fynn Baumgartner, Leon Blum*

*Research: Fynn Baumgartner, Leon Blum*

## Project Scope

This project aims to optimize a pretrained model on the EuroSAT_RGB dataset [1], with the goal of determining how small the model can be made while maintaining reasonable accuracy. To this end, two pretrained architectures will be selected — one larger than 10 MB and one smaller than 10 MB — and optimized using pruning and quantization. This will also allow us to study whether a correlation exists between optimization effectiveness (e.g., pruning and quantization) and model parameter count.

# Table of Contents

Table of Contents ..... 2
1. Model Comparison ..... 3
1.1 Initial Comparison ..... 3
1.2 Adjusting the Comparison Approach Based on Previous Findings ..... 3
2. Size Optimization of Selected Models ..... 7
2.1 Pruning ..... 7
2.2 Quantization ..... 11
MobileNetV3-Small ..... 11
MobileNetV3-Large ..... 11
Quantization while keeping SE blocks in fp32 ..... 12
Quantization while keeping HS blocks in fp32 ..... 13
2.3 Combining Pruning and Quantization ..... 15
MobileNetV3-Small ..... 15
MobileNetV3-Large ..... 15
3. Conclusion ..... 15
References ..... 15

# 1. Model Comparison

*Note: The early screening and hyperparameter-selection runs (Sections 1.1–1.2) were single-seeded runs used to guide architecture and hyperparameter decisions. Full determinism was not feasible given the available computational resources. It was expected that the differences between models would be large enough to outweigh run-to-run variance.*

## 1.1 Initial Comparison

**Goal:** Identify the two models best suited to the chosen dataset (EuroSAT_RGB) — one larger than 10 MB and one smaller than 10 MB.

**Methodology:** Compare four model architectures under fixed training parameters.

**Training parameters:**

- Patience: 5
- Min delta: 0.001
- Learning rate: 0.001
- Training set size: 0.7 × dataset
- Precision: fp32
- Bounded at 100 epochs
- Fixed manual seed

**Results:**

| Model | Best Val. Loss | Best Val. Acc. | Total Params | Trainable Params | Size (MB) |
|---|---|---|---|---|---|
| MobileNetV3-Small | 0.0827 | 0.9795 | 1,528,106 | 1,528,106 | 5.88 |
| MobileNetV3-Large | 0.1142 | 0.9674 | 4,214,842 | 4,214,842 | 16.17 |
| EfficientNet-Lite | 0.0691 | 0.9812 | 3,383,818 | 3,383,818 | 13.07 |
| Ultralytics YOLOv8 | 0.0937 | 0.9696 | 1,451,098 | 1,451,098 | 5.56 |

*Note: YOLOv8's classification head returns softmax probabilities rather than logits in evaluation mode, which initially corrupted the validation loss (~1.49). This was fixed by patching the head to always return raw logits; the table above reflects the corrected values.*

**Evaluation:** MobileNetV3-Small [2] and YOLOv8 [3] achieve high accuracy and low loss relative to their size, but both have fewer trainable parameters, leaving less room for further size optimization such as pruning. EfficientNet-Lite [4] delivers the best overall performance, but at 2–3 times the size of MobileNetV3-Small and YOLOv8. Surprisingly, MobileNetV3-Large underperformed, being outperformed by the smaller model in every metric. This suggests that comparing models under identical training parameters is insufficient, as different architectures likely require different learning rates or patience values.

**Conclusion:** These results are inconclusive. Additional information on model convergence is required.

## 1.2 Adjusting the Comparison Approach Based on Previous Findings

It became clear that a single comparison under fixed training parameters was insufficient to fairly evaluate the different models. Given our computational constraints, two changes were made to make the comparison more feasible:

- Training set size: reduced from 0.7 × dataset to 0.35 × dataset
- Precision: changed from single precision (fp32) to automatic mixed precision (fp16)

To evaluate model convergence, we now plot the validation loss per epoch.

**Training parameters:**

- Patience: 5
- Min delta: 0.001
- Learning rate: 0.001
- Training set size: 0.35 × dataset
- Precision: fp16
- Bounded at 100 epochs
- Fixed manual seed

**Results:**

| Model | Best Val. Loss | Best Val. Acc. | Total Params | Trainable Params | Size (MB) |
|---|---|---|---|---|---|
| MobileNetV3-Small | 0.0985 | 0.9732 | 1,528,106 | 1,528,106 | 5.88 |
| MobileNetV3-Large | 0.0959 | 0.9741 | 4,214,842 | 4,214,842 | 16.17 |
| EfficientNet-Lite | 0.0834 | 0.9752 | 3,383,818 | 3,383,818 | 13.07 |
| Ultralytics YOLOv8 | 0.1055 | 0.9690 | 1,451,098 | 1,451,098 | 5.56 |

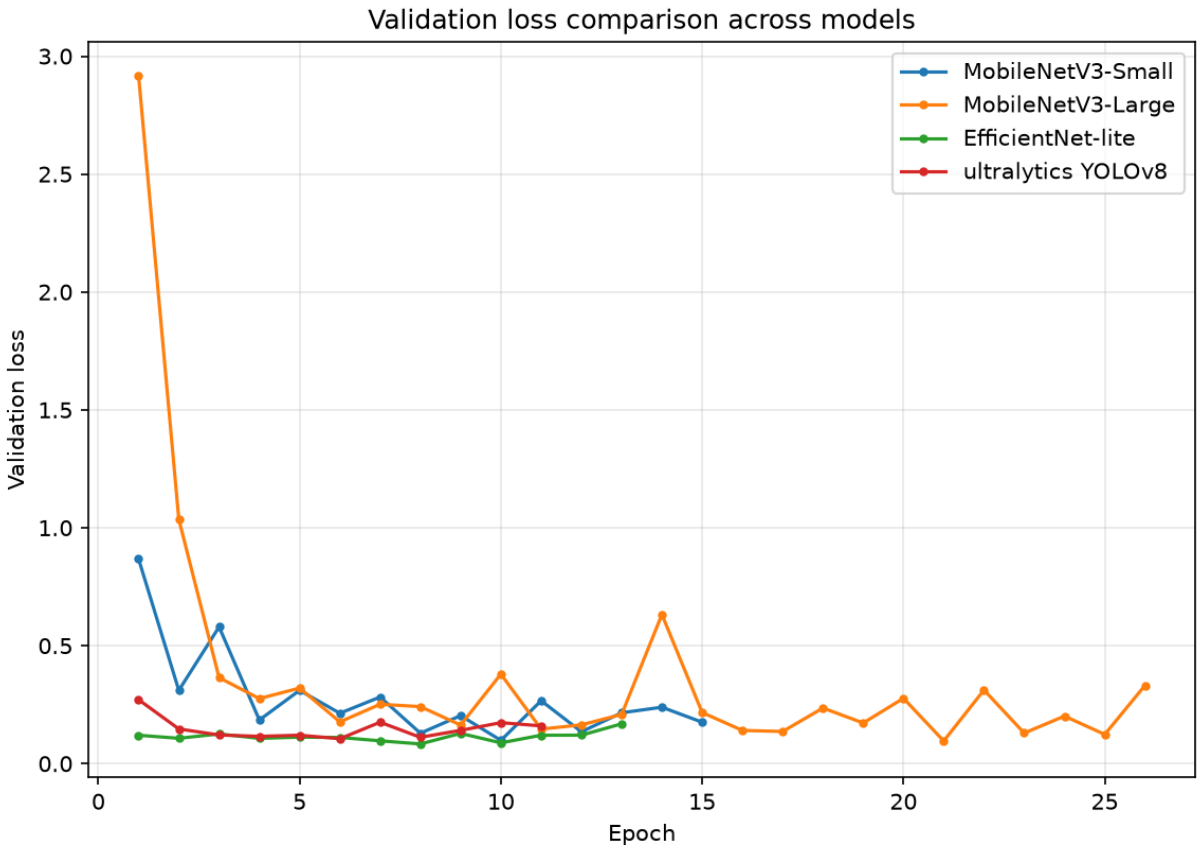



*Figure 1. Validation loss per epoch — initial fp16 run.*

**Evaluation:** The plots from the second run confirm that MobileNetV3-Large has not yet converged. The oscillations suggest an overly aggressive learning rate. MobileNetV3-Small shows a similar but much smaller effect. YOLOv8 and EfficientNet-Lite appear to converge initially but begin overfitting after epochs 6–8.

**Adjustments:**

MobileNetV3-Small & MobileNetV3-Large:

- Learning rate reduced from 0.001 to 0.0002 to improve convergence

YOLOv8 & EfficientNet-Lite:

- As both converge adequately, overfitting was addressed by reducing patience from 5 to 3

**Results:**

| Model | Best Val. Loss | Best Val. Acc. | Total Params | Trainable Params | Size (MB) |
|---|---|---|---|---|---|
| MobileNetV3-Small | 0.0755 | 0.9753 | 1,528,106 | 1,528,106 | 5.88 |
| MobileNetV3-Large | 0.0787 | 0.9780 | 4,214,842 | 4,214,842 | 16.17 |
| EfficientNet-Lite | 0.0963 | 0.9715 | 3,383,818 | 3,383,818 | 13.07 |
| Ultralytics YOLOv8 | 0.1120 | 0.9631 | 1,451,098 | 1,451,098 | 5.56 |

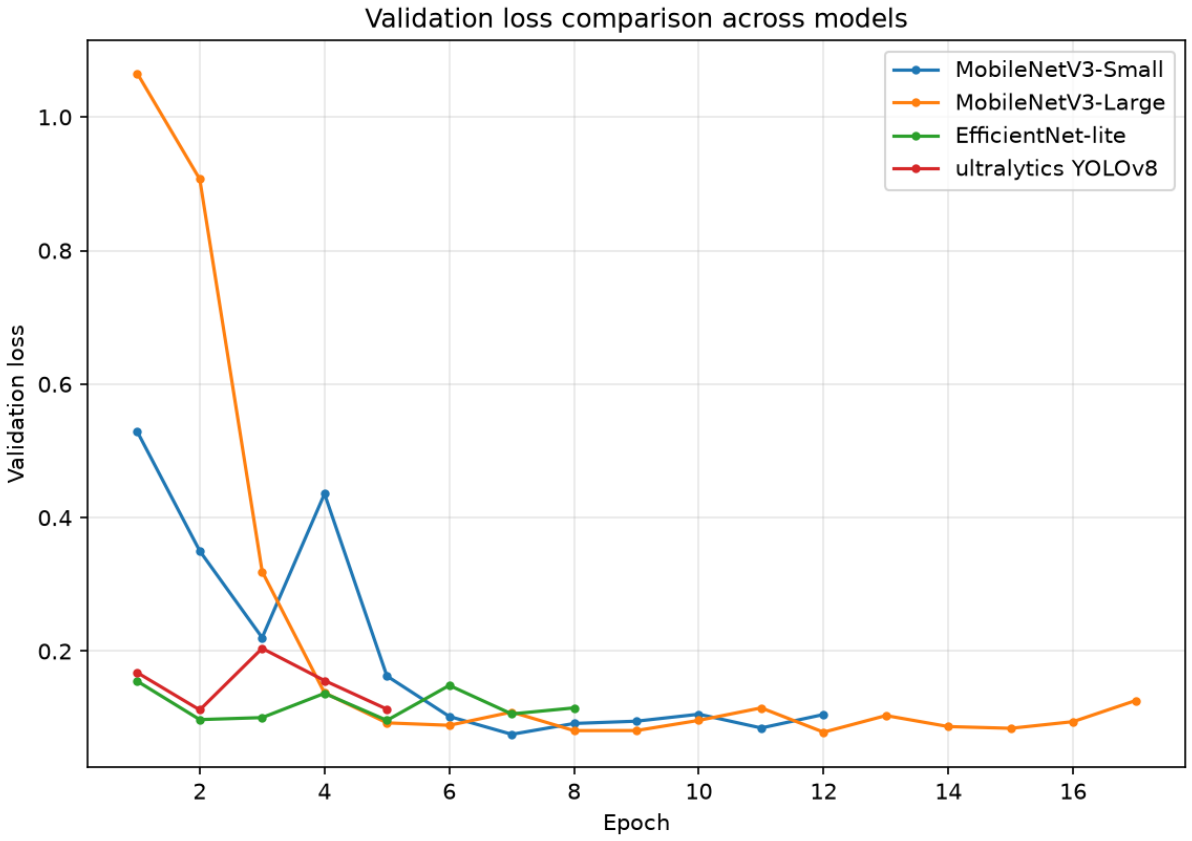



*Figure 2. Validation loss per epoch after learning-rate and patience adjustments.*

**Evaluation:** The new results show a clear improvement in convergence for the MobileNetV3 architectures. EfficientNet-Lite shows convergence behavior similar to the previous run. YOLOv8's convergence, however, degraded compared to the previous run. Both YOLOv8 and EfficientNet-Lite still lack sufficient convergence for a fair comparison.

To further improve convergence for YOLOv8 and EfficientNet-Lite, the following parameters were tested:

- Learning rate: 0.0005
- Patience: 4

**Results:**

| Model | Best Val. Loss | Best Val. Acc. | Total Params | Trainable Params | Size (MB) |
|---|---|---|---|---|---|
| EfficientNet-Lite | 0.0840 | 0.9783 | 3,383,818 | 3,383,818 | 13.07 |
| Ultralytics YOLOv8 | 0.0858 | 0.9747 | 1,451,098 | 1,451,098 | 5.56 |

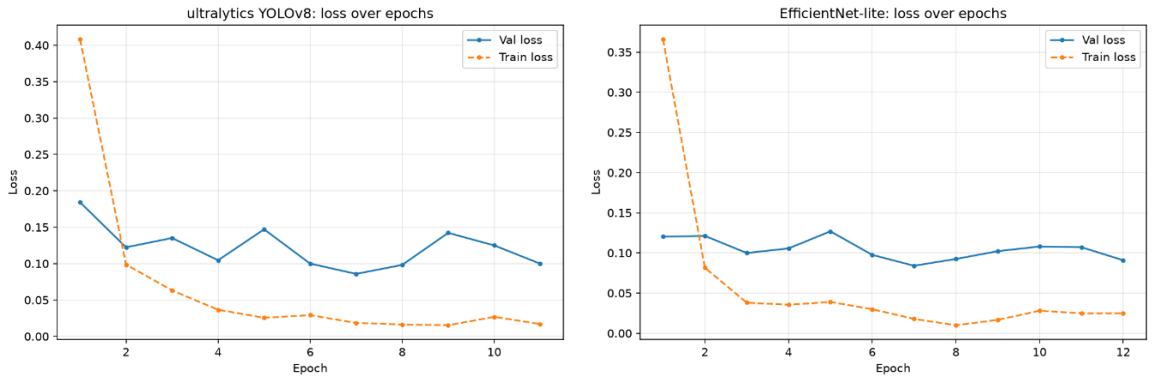



*Figure 3. Validation loss per epoch for EfficientNet-Lite (left) and YOLOv8 (right) after the final adjustment.*

**Evaluation:** Both YOLOv8 and EfficientNet-Lite show significantly improved convergence. YOLOv8 still exhibits oscillating behavior and would likely benefit from a further reduction in learning rate.

**Comparison:**

| Model | Best Val. Loss | Best Val. Acc. | Total Params | Trainable Params | Size (MB) |
|---|---|---|---|---|---|
| MobileNetV3-Small | 0.0755 | 0.9753 | 1,528,106 | 1,528,106 | 5.88 |
| MobileNetV3-Large | 0.0787 | 0.9780 | 4,214,842 | 4,214,842 | 16.17 |
| EfficientNet-Lite | 0.0840 | 0.9783 | 3,383,818 | 3,383,818 | 13.07 |
| Ultralytics YOLOv8 | 0.0858 | 0.9747 | 1,451,098 | 1,451,098 | 5.56 |

**Conclusion:** Contrary to expectations, all four models adapted similarly well to the dataset. The observed differences are likely attributable to how well the training parameters were tuned for each model. Adjusting hyperparameters yielded up to ~28% improvement in validation loss, while the final differences between models reached only ~12%. In addition, the original assumption that differences in model performance would outweigh run-to-run variance could not be confirmed; further testing would be required.

This means that, at this stage, a performance-based comparison between the four models would not be well-founded.

It appears that hyperparameter tuning has a substantially larger effect on model performance than how well the model architecture fits this particular dataset.

Investigating the effect of architecture–dataset fit would constitute a research topic in its own right. Since this project focuses on optimizing models for size while maintaining accuracy — and the four-architecture comparison was originally intended only as an inexpensive initial filter — we will not pursue this line of research further.

For readers interested in this question, possible next steps include:

1. Developing a hyperparameter-tuning algorithm in Python to achieve good convergence across all models.
2. Studying the effect of run-to-run variance by averaging results from the previously identified configurations across multiple iterations.

# 2. Size Optimization of Selected Models

Since no clearly best-fitting model architecture emerged from the model comparison, we chose to continue optimizing MobileNetV3-Large and MobileNetV3-Small, for the following reasons:

- Both are based on the MobileNet architecture, allowing us to study the correlation between pruning effectiveness and parameter count more effectively than would be possible with two entirely different architectures.
- Both showed promising convergence under the same hyperparameters, allowing the following optimization to be conducted under fixed conditions (learning rate: 0.0002, patience: 5), reducing the variance that hyperparameter tuning introduced into the earlier results.

## 2.1 Pruning

The first optimization method we chose was pruning. Pretrained models like MobileNetV3 carry more capacity than our task requires: they are designed for ImageNet-scale classification with 1,000 output classes [2], whereas our task distinguishes only the 10 EuroSAT classes [1]. Pruning removes the parameters that contribute least to the task and therefore directly addresses our goal of reducing the model to the required size. In detail, we chose structured pruning. We decided not to pursue unstructured pruning: it produces sparse weight matrices, and while specialized hardware can reduce compute by skipping the resulting zero weights, most hardware cannot, and the sparse matrices do not physically reduce the size of the model [5], [6]. Structured pruning, in contrast, removes entire channels and filters, allowing us not only to reduce the model's physical size but, in some cases, to improve inference performance, though this depends on the pruning method, hardware, and model [6], [7].

**Methodology:**

Both models were trained with the above-mentioned parameters. The base models are shown in Table 1 (ratio 0).

Our first step was figuring out how to perform structured pruning while preserving the model's structure. We faced the issue that removing a single layer or parameter might cause a model mismatch, as layers are often interdependent. To address this problem, we used the Torch-Pruning library [8], which implements the DepGraph algorithm [9]. DepGraph builds a dependency graph over the network to group interdependent layers, so that when a channel is pruned, all coupled parameters are removed together, leaving the architecture intact [9]. The importance of each group is determined via the L2 norm of its weights, and depending on the pruning ratio, the least important groups are removed [8], [9].

Following pruning, performance is often severely degraded because parts of the model's structure have been removed. We therefore retrained the pruned models with the same parameters as before (learning rate = 0.0002, patience = 5, min delta = 0.001), following the common prune-and-retrain approach [5]. To test the models' behavior across different pruning ratios, we ran an algorithm that performed a one-time prune followed by retraining, and used the macro-F1 score to check for uneven performance across the 10 classes. Both the Small and Large versions of MobileNetV3 responded well to pruning; 12–15 epochs were needed in most cases for the models to converge again. The validation accuracy showed that, with increasing pruning ratio, accuracy decreases, as expected. Here, though, the pruning ratio of 0.9 proved to be an outlier. It had, as expected, the smallest parameter count, but its MAC count increased compared to the previous models, and it showed higher accuracy than the immediately preceding pruning ratio for both the Small and Large models. The 0.9 models benefited from more epochs, as they did not trigger the min-delta criterion as early as the previous models, but a definitive answer as to why the MAC count increased could not be found. One plausible explanation involves the round_to parameter of Torch-Pruning, which we set to 8 so that channel counts remain multiples of 8 for hardware efficiency [8]. With the model shrinking so aggressively at the 0.9 pruning ratio, the requested channel counts fall below this floor in many layers and are rounded back up to 8, which would explain the increase in MACs. This, however, needs more rigorous testing of the parameters, which is beyond the scope of this project and is left to the interested reader.

Pruning a model aggressively in a single step, especially at high ratios, tends to cause a larger drop in performance, since many important channels are removed at once and their function cannot be redistributed onto the remaining parameters [5], [7]. To test whether spreading the pruning over several steps mitigates this, we compared the large model at a pruning ratio of 0.5 in a one-shot and an iterative process. At 3 iterative steps, the model performed slightly worse than the one-shot baseline; at 10 steps, we observed a small accuracy gain of roughly 2%. With only a single run per condition, however, a difference of this size cannot be separated from ordinary run-to-run variance, which can arise from random weight initialization, data-split ordering, and dropout alone. Confirming whether the iterative gain is real would require repeating each condition across several random seeds and checking that the gap exceeds the variance between runs. We therefore treat this result as suggestive rather than conclusive.

**Summary:**

Structured pruning shows high potential for running on edge machines, as we were able to reduce the model size by a large margin while maintaining high accuracy.

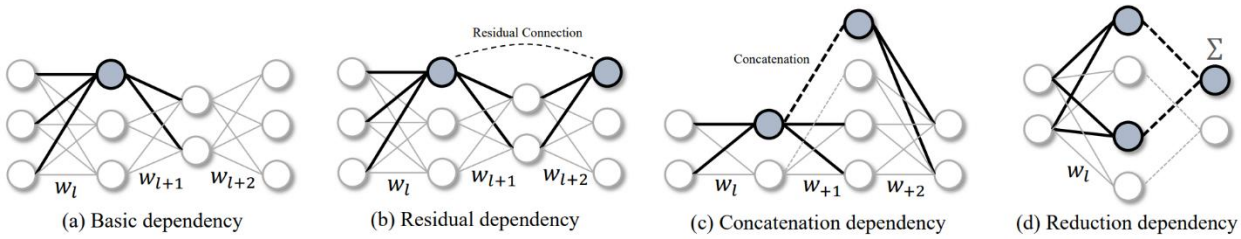



Figure 2. Grouped parameters with inter-dependency in different structures. All highlighted parameters must be pruned simultaneously.

Table 1. Absolute metrics

| Model | Ratio | File (MB) | Mem (MB) | Params (M) | MACs (G) | Accuracy | Macro-F1 |
|---|---|---|---|---|---|---|---|
| Large | 0 | 16.279 | 16.172 | 4.2148 | 0.2271 | 0.9777 | 0.9765 |
| Large | 0.1 | 13.121 | 12.969 | 3.3779 | 0.1664 | 0.9491 | 0.9472 |
| Large | 0.2 | 10.516 | 10.363 | 2.6972 | 0.1398 | 0.9347 | 0.9323 |
| Large | 0.3 | 8.064 | 7.911 | 2.057 | 0.1073 | 0.9228 | 0.9194 |
| Large | 0.4 | 6.045 | 5.893 | 1.5304 | 0.0816 | 0.8904 | 0.8857 |
| Large | 0.5 | 4.301 | 4.149 | 1.0756 | 0.0613 | 0.893 | 0.8894 |
| Large | 0.6 | 2.842 | 2.689 | 0.6952 | 0.0528 | 0.8956 | 0.8898 |
| Large | 0.7 | 1.714 | 1.562 | 0.4021 | 0.0398 | 0.8822 | 0.8778 |
| Large | 0.8 | 0.792 | 0.641 | 0.1633 | 0.027 | 0.8695 | 0.8642 |
| Large | 0.9 | 0.447 | 0.296 | 0.0735 | 0.0649 | 0.9159 | 0.9125 |
| Small | 0 | 5.961 | 5.876 | 1.5281 | 0.0591 | 0.9562 | 0.9542 |
| Small | 0.1 | 4.718 | 4.596 | 1.1942 | 0.0407 | 0.9444 | 0.9417 |
| Small | 0.2 | 3.789 | 3.667 | 0.9519 | 0.0346 | 0.9286 | 0.9253 |
| Small | 0.3 | 2.946 | 2.824 | 0.7321 | 0.0284 | 0.9198 | 0.9154 |
| Small | 0.4 | 2.218 | 2.098 | 0.5428 | 0.0222 | 0.9063 | 0.9007 |
| Small | 0.5 | 1.661 | 1.54 | 0.3976 | 0.0175 | 0.8964 | 0.8925 |
| Small | 0.6 | 1.079 | 0.956 | 0.2459 | 0.0175 | 0.9351 | 0.9324 |
| Small | 0.7 | 0.675 | 0.554 | 0.1417 | 0.0146 | 0.9323 | 0.9294 |
| Small | 0.8 | 0.421 | 0.3 | 0.0761 | 0.0128 | 0.9105 | 0.9063 |
| Small | 0.9 | 0.271 | 0.151 | 0.0377 | 0.0162 | 0.9301 | 0.9281 |

*File = on-disk .pth size; Mem = in-memory size; MACs = multiply-accumulates per forward pass;*

Table 2. Change versus unpruned baseline

| Model | Ratio | File % | Mem % | Params % | MACs % | Acc Δpp | F1 Δpp |
|---|---|---|---|---|---|---|---|
| Large | 0 | — | — | — | — | — | — |
| Large | 0.1 | -19.4 | -19.81 | -19.86 | -26.72 | -2.85 | -2.93 |
| Large | 0.2 | -35.4 | -35.92 | -36.01 | -38.45 | -4.3 | -4.42 |
| Large | 0.3 | -50.46 | -51.08 | -51.2 | -52.74 | -5.48 | -5.71 |
| Large | 0.4 | -62.86 | -63.56 | -63.69 | -64.05 | -8.73 | -9.08 |
| Large | 0.5 | -73.58 | -74.34 | -74.48 | -73 | -8.47 | -8.71 |
| Large | 0.6 | -82.54 | -83.37 | -83.51 | -76.74 | -8.21 | -8.67 |
| Large | 0.7 | -89.47 | -90.34 | -90.46 | -82.48 | -9.54 | -9.87 |
| Large | 0.8 | -95.13 | -96.03 | -96.13 | -88.1 | -10.81 | -11.23 |
| Large | 0.9 | -97.26 | -98.17 | -98.26 | -71.42 | -6.17 | -6.4 |
| Small | 0 | — | — | — | — | — | — |
| Small | 0.1 | -20.84 | -21.77 | -21.85 | -31.17 | -1.17 | -1.24 |
| Small | 0.2 | -36.43 | -37.58 | -37.71 | -41.44 | -2.75 | -2.89 |
| Small | 0.3 | -50.57 | -51.93 | -52.09 | -51.89 | -3.64 | -3.87 |
| Small | 0.4 | -62.79 | -64.3 | -64.48 | -62.47 | -4.99 | -5.35 |
| Small | 0.5 | -72.13 | -73.8 | -73.98 | -70.34 | -5.98 | -6.17 |
| Small | 0.6 | -81.9 | -83.73 | -83.91 | -70.42 | -2.11 | -2.17 |
| Small | 0.7 | -88.68 | -90.56 | -90.72 | -75.25 | -2.38 | -2.47 |
| Small | 0.8 | -92.93 | -94.89 | -95.02 | -78.37 | -4.57 | -4.79 |
| Small | 0.9 | -95.45 | -97.43 | -97.53 | -72.63 | -2.6 | -2.61 |

*File/Mem/Params/MACs vs the ratio-0.0 baseline of the same model. Accuracy and Macro-F1 as change in percentage points (Δpp). Negative = smaller/faster/lower.*

## 2.2 Quantization

A different model-compression technique is quantization, which reduces the precision of weight and activation tensors relative to the precision used during training. Reducing precision from 32-bit to 8-bit, for example, can reduce memory overhead by a factor of 4 and computational cost by a factor of 16 [10].

In this project we use post-training quantization (PTQ) algorithms, as they require little to no hyperparameter tuning, making them an efficient way to quantize a neural network [10].

**Methodology:** We take the unpruned MobileNetV3 models trained earlier and quantize them from 32-bit to 8-bit precision. For each quantized model, we benchmark size, loss, and accuracy against the baseline.

We use Quantization – PTQ using PyTorch 2 Export Quantization and X86 Backend [11], since the models are developed on an x86 CPU architecture. For deployment on mobile devices with different architectures, Quantization – PTQ using PyTorch 2 Export Quantization and XNNPACK Quantizer [12] should be used instead.

**Setup:**

- Model: MobileNetV3 (benchmark version; learning rate 0.0002, patience 5)
- Calibration dataset: training dataset (Section 1.2)
- Evaluation dataset: evaluation dataset (Section 1.2)

**Results:**

### MobileNetV3-Small

| Metric | Baseline (fp32) | Quantized (Int8) |
|---|---|---|
| Loss | 0.0808 | 5.2002 |
| Accuracy | 0.9737 | 0.1346 |
| Total unquantized parameters | 1,528,106 | 9,978 |
| Trainable unquantized parameters | 1,528,106 | 3,922 |
| Size (MB) | 5.88 | 1.59 |

### MobileNetV3-Large

| Metric | Baseline (fp32) | Quantized (Int8) |
|---|---|---|
| Loss | 0.0829 | 0.5589 |
| Accuracy | 0.9757 | 0.8544 |
| Total unquantized parameters | 4,214,842 | 18,570 |
| Trainable unquantized parameters | 4,214,842 | 6,370 |
| Size (MB) | 16.17 | 4.27 |

MobileNetV3-Small degrades to near-random decision-making after quantization. MobileNetV3-Large, in contrast, while still showing a substantial increase in loss, retains an acceptable accuracy of 0.85 despite shrinking by almost 12 MB.

This raises the question of why MobileNetV3-Small is far more susceptible to quantization noise than MobileNetV3-Large.

Comparing the architectures side by side (Figure 4), three main differences stand out:

| Input | Operator | exp size | $\#out$ | SE | NL | $s$ |
|---|---|---|---|---|---|---|
| $224^2 \times 3$ | conv2d | - | 16 | - | HS | 2 |
| $112^2 \times 16$ | bneck, 3x3 | 16 | 16 | - | RE | 1 |
| $112^2 \times 16$ | bneck, 3x3 | 64 | 24 | - | RE | 2 |
| $56^2 \times 24$ | bneck, 3x3 | 72 | 24 | - | RE | 1 |
| $56^2 \times 24$ | bneck, 5x5 | 72 | 40 | ✓ | RE | 2 |
| $28^2 \times 40$ | bneck, 5x5 | 120 | 40 | ✓ | RE | 1 |
| $28^2 \times 40$ | bneck, 5x5 | 120 | 40 | ✓ | RE | 1 |
| $28^2 \times 40$ | bneck, 3x3 | 240 | 80 | - | HS | 2 |
| $14^2 \times 80$ | bneck, 3x3 | 200 | 80 | - | HS | 1 |
| $14^2 \times 80$ | bneck, 3x3 | 184 | 80 | - | HS | 1 |
| $14^2 \times 80$ | bneck, 3x3 | 184 | 80 | - | HS | 1 |
| $14^2 \times 80$ | bneck, 3x3 | 480 | 112 | ✓ | HS | 1 |
| $14^2 \times 112$ | bneck, 3x3 | 672 | 112 | ✓ | HS | 1 |
| $14^2 \times 112$ | bneck, 5x5 | 672 | 160 | ✓ | HS | 2 |
| $7^2 \times 160$ | bneck, 5x5 | 960 | 160 | ✓ | HS | 1 |
| $7^2 \times 160$ | bneck, 5x5 | 960 | 160 | ✓ | HS | 1 |
| $7^2 \times 160$ | conv2d, 1x1 | - | 960 | - | HS | 1 |
| $7^2 \times 960$ | pool, 7x7 | - | - | - | - | 1 |
| $1^2 \times 960$ | conv2d 1x1, NBN | - | 1280 | - | HS | 1 |
| $1^2 \times 1280$ | conv2d 1x1, NBN | - | k | - | - | 1 |

| Input | Operator | exp size | $\#out$ | SE | NL | $s$ |
|---|---|---|---|---|---|---|
| $224^2 \times 3$ | conv2d, 3x3 | - | 16 | - | HS | 2 |
| $112^2 \times 16$ | bneck, 3x3 | 16 | 16 | ✓ | RE | 2 |
| $56^2 \times 16$ | bneck, 3x3 | 72 | 24 | - | RE | 2 |
| $28^2 \times 24$ | bneck, 3x3 | 88 | 24 | - | RE | 1 |
| $28^2 \times 24$ | bneck, 5x5 | 96 | 40 | ✓ | HS | 2 |
| $14^2 \times 40$ | bneck, 5x5 | 240 | 40 | ✓ | HS | 1 |
| $14^2 \times 40$ | bneck, 5x5 | 240 | 40 | ✓ | HS | 1 |
| $14^2 \times 40$ | bneck, 5x5 | 120 | 48 | ✓ | HS | 1 |
| $14^2 \times 48$ | bneck, 5x5 | 144 | 48 | ✓ | HS | 1 |
| $14^2 \times 48$ | bneck, 5x5 | 288 | 96 | ✓ | HS | 2 |
| $7^2 \times 96$ | bneck, 5x5 | 576 | 96 | ✓ | HS | 1 |
| $7^2 \times 96$ | bneck, 5x5 | 576 | 96 | ✓ | HS | 1 |
| $7^2 \times 96$ | conv2d, 1x1 | - | 576 | ✓ | HS | 1 |
| $7^2 \times 576$ | pool, 7x7 | - | - | - | - | 1 |
| $1^2 \times 576$ | conv2d 1x1, NBN | - | 1024 | - | HS | 1 |
| $1^2 \times 1024$ | conv2d 1x1, NBN | - | k | - | - | 1 |

*Figure 4. The MobileNetV3-Large (left) and MobileNetV3-Small (right) architectures. Source: Howard et al. [2].*

- MobileNetV3-Large has a substantially higher parameter count than MobileNetV3-Small (~4 million vs. ~1.5 million).
- MobileNetV3-Large contains significantly more 3×3 bottleneck (bneck) layers.
- MobileNetV3-Small has a significantly higher proportion of blocks containing a Squeeze-and-Excitation (SE) mechanism.
- MobileNetV3-Small has a significantly higher proportion of blocks using the Hard-Swish (HS) activation function.

The 3×3 bneck layers can be ruled out as the source of the pronounced degradation, since they are the dominant layer type in both networks; if they were responsible, we would expect similar degradation in both models. This leaves parameter count and the prevalence of SE/HS blocks as the most likely sources of the difference in quantization sensitivity.

Both “FrostNet: Towards Quantization-Aware Network Architecture Search” [13] and the TensorFlow blog post “Higher accuracy on vision models with EfficientNet-Lite” [4] suggest that removing SE and HS blocks improves post-training quantization results. We therefore repeated quantization on both models, once excluding all SE blocks and once excluding all HS blocks from quantization.

## Quantization while keeping SE blocks in fp32

This produces a hybrid-precision model, with all non-SE blocks quantized to int8 and all SE blocks kept in fp32.

**MobileNetV3-Small**

| Metric | Baseline (fp32) | Quantized (Int8) |
|---|---|---|
| Loss | 0.0808 | 4.9842 |
| Accuracy | 0.9737 | 0.1495 |
| Total unquantized parameters | 1,528,106 | 468,730 |
| Trainable unquantized parameters | 1,528,106 | 462,674 |
| Size (MB) | 5.88 | 2.87 |

**MobileNetV3-Large**

| Metric | Baseline (fp32) | Quantized (Int8) |
|---|---|---|
| Loss | 0.0829 | 0.5647 |
| Accuracy | 0.9757 | 0.8521 |
| Total unquantized parameters | 4,214,842 | 1,525,770 |
| Trainable unquantized parameters | 4,214,842 | 1,513,570 |
| Size (MB) | 16.17 | 8.53 |

## Quantization while keeping HS blocks in fp32

This produces a hybrid-precision model, with all non-HS blocks quantized to int8 and all HS blocks kept in fp32.

**MobileNetV3-Small**

| Metric | Baseline (fp32) | Quantized (Int8) |
|---|---|---|
| Loss | 0.0808 | 5.2003 |
| Accuracy | 0.9737 | 0.1306 |
| Total unquantized parameters | 1,528,106 | 880,402 |
| Trainable unquantized parameters | 1,528,106 | 874,346 |
| Size (MB) | 5.88 | 4.01 |

**MobileNetV3-Large**

| Metric | Baseline (fp32) | Quantized (Int8) |
|---|---|---|
| Loss | 0.0829 | 0.2607 |
| Accuracy | 0.9757 | 0.9185 |
| Total unquantized parameters | 4,214,842 | 2,047,810 |
| Trainable unquantized parameters | 4,214,842 | 2,035,610 |
| Size (MB) | 16.17 | 9.95 |

**Interpretation:** MobileNetV3-Small remains non-functional regardless of the applied changes, supporting the hypothesis that resilience to quantization noise scales with overall parameter count. MobileNetV3-Large shows almost no response to excluding SE layers, aside from an increase in size due to the additional fp32 layers — a surprising result, since it implies the exclusion made the model perform slightly worse. Excluding HS layers, however, significantly improved MobileNetV3-Large's performance, though this may partly reflect the larger number of layers set to fp32 relative to the fully quantized model.

To further investigate the disproportionate degradation of MobileNetV3-Small, we performed block-by-block quantization using two tests:

3. Leave-one-out (LOO): the entire model is set to int8, and each block is individually restored to fp32.
4. Leave-one-in (LOI): the entire model is set to fp32, and each block is individually set to int8.

LOI measures each block's contribution to accuracy degradation (“damage”), while LOO measures the accuracy recovered when a given block is restored to fp32 (“recovery”). Together, these tests aim to isolate the specific architectural characteristics responsible for the disproportionate loss in accuracy.

| candidate | fp32_params | granularity | loi_acc | loi_acc_damage | loi_loss | loo_acc | loo_acc_recovered | loo_loss | module_names |
|---|---|---|---|---|---|---|---|---|---|
| block_features.1 | 744 | block | 0.16679012 | 0.80691358 | 4.4515969 | 0.94061728 | 0.806296296 | 0.18948099 | features.1 |
| block_features.5 | 57264 | block | 0.9745679 | -0.000864198 | 0.08037013 | 0.14888889 | 0.014567901 | 5.01331708 | features.5 |
| block_features.2 | 3864 | block | 0.93950617 | 0.034197531 | 0.19753613 | 0.1482716 | 0.013950617 | 4.85323333 | features.2 |
| SE_features.5.block.2 | 31024 | layer_type_se | 0.97395062 | -0.000246914 | 0.08078224 | 0.14617284 | 0.011851852 | 5.04763656 | features.5.block.2 |
| SE_features.10.block.2 | 166608 | layer_type_se | 0.97395062 | -0.000246914 | 0.08074746 | 0.13790123 | 0.003580247 | 5.06553984 | features.10.block.2 |
| block_features.0 | 464 | block | 0.9745679 | -0.000864198 | 0.08067587 | 0.13666667 | 0.002345679 | 5.03311711 | features.0 |
| SE_features.11.block.2 | 166608 | layer_type_se | 0.97395062 | -0.000246914 | 0.08076467 | 0.1362963 | 0.001975309 | 5.06170982 | features.11.block.2 |
| block_features.9 | 91848 | block | 0.97395062 | -0.000246914 | 0.08026175 | 0.13592593 | 0.001604938 | 5.13672926 | features.9 |
| block_features.4 | 13736 | block | 0.9737037 | 0 | 0.08328633 | 0.13555556 | 0.001234568 | 5.08296511 | features.4 |
| block_features.11 | 294096 | block | 0.97358025 | 0.000123457 | 0.08026045 | 0.13555556 | 0.001234568 | 5.11072056 | features.11 |
| SE_features.9.block.2 | 41832 | layer_type_se | 0.97395062 | -0.000246914 | 0.08081109 | 0.13530864 | 0.000987654 | 5.12461954 | features.9.block.2 |
| block_features.3 | 5416 | block | 0.9737037 | 0 | 0.08430176 | 0.13506173 | 0.000740741 | 5.05145067 | features.3 |
| SE_features.4.block.2 | 4728 | layer_type_se | 0.9737037 | 0 | 0.08074151 | 0.13432099 | 0 | 5.15223035 | features.4.block.2 |
| block_features.12 | 56448 | block | 0.97382716 | -0.000123457 | 0.08083551 | 0.13407407 | -0.000246914 | 5.12730583 | features.12 |
| block_features.6 | 57264 | block | 0.97493827 | -0.001234568 | 0.0814416 | 0.13358025 | -0.000740741 | 5.09809672 | features.6 |
| block_features.8 | 29800 | block | 0.97395062 | -0.000246914 | 0.08099849 | 0.13320988 | -0.001111111 | 5.14254702 | features.8 |
| SE_features.7.block.2 | 7832 | layer_type_se | 0.97395062 | -0.000246914 | 0.08077155 | 0.13320988 | -0.001111111 | 5.08231405 | features.7.block.2 |
| SE_features.8.block.2 | 11704 | layer_type_se | 0.9737037 | 0 | 0.08056381 | 0.13259259 | -0.001728395 | 5.15808261 | features.8.block.2 |
| block_features.10 | 294096 | block | 0.97395062 | -0.000246914 | 0.08086681 | 0.13246914 | -0.001851852 | 5.09367661 | features.10 |
| SE_features.1.block.1 | 280 | layer_type_se | 0.9737037 | 0 | 0.0807744 | 0.13234568 | -0.001975309 | 5.09855524 | features.1.block.1 |
| SE_features.6.block.2 | 31024 | layer_type_se | 0.97382716 | -0.000123457 | 0.08079268 | 0.13185185 | -0.002469136 | 5.12825918 | features.6.block.2 |
| block_features.7 | 21968 | block | 0.97419753 | -0.000493827 | 0.08094217 | 0.13098765 | -0.003333333 | 5.13604795 | features.7 |

*Figure 5. MobileNetV3-Small block-wise LOO/LOI results, sorted by LOO recovery.*

**Evaluation:** The results clearly show that the features.1 block is responsible for MobileNetV3-Small's degradation, reducing accuracy by roughly 80 percentage points on its own when quantized. This directly disproves our earlier hypothesis that SE and HS blocks were responsible for the degradation.

Based on these results, MobileNetV3-Small can be quantized with minimal accuracy loss simply by keeping the features.1 block in fp32.

| Metric | Baseline (fp32) | Quantized Int8 (features.1 fp32) |
|---|---|---|
| Loss | 0.0808 | 0.1894 |
| Accuracy | 0.9737 | 0.9420 |
| Total unquantized parameters | 1,528,106 | 10,634 |
| Trainable unquantized parameters | 1,528,106 | 4,578 |
| Size (MB) | 5.88 | 1.60 |

Comparing this directly with the fully quantized model:

| Metric | Quantized Int8 | Quantized Int8 (features.1 fp32) |
|---|---|---|
| Loss | 5.2002 | 0.1894 |
| Accuracy | 0.1346 | 0.9420 |
| Total unquantized parameters | 9,978 | 10,634 |
| Trainable unquantized parameters | 3,922 | 4,578 |
| Size (MB) | 1.59 | 1.60 |

Keeping a single block in fp32 increases model size only marginally, from 1.59 MB to 1.60 MB, while increasing accuracy from 0.1346 to 0.9420.

**Conclusion:** A specific mechanism within MobileNetV3-Small's features.1 block appears to be responsible for nearly all of the accuracy degradation caused by quantization. Due to time constraints and the increasing complexity of this line of investigation, we did not explore this phenomenon further within the scope of this project.

For readers interested in pursuing this question, possible next steps include:

5. Examining the internal components of features.1 in more detail and running a second LOO/LOI analysis restricted to that block.
6. Comparing the building blocks of features.1 with those of the remaining blocks in MobileNetV3-Small to identify characteristics unique to this block.

### 2.3 Combining Pruning and Quantization

In this step, we combine our most effective quantization method with our most successful pruning configuration to obtain a final, fully optimized model for each architecture.

#### MobileNetV3-Small

**Method:** One-time structured pruning at a 90% pruning ratio, followed by int8 quantization excluding the features.1 block.

| Metric | Baseline | Optimized |
|---|---|---|
| Loss | 0.0808 | 0.3815 |
| Accuracy | 0.9737 | 0.8764 |
| Total unquantized parameters | 1,528,106 | 2,226 |
| Trainable unquantized parameters | 1,528,106 | 1,260 |
| Size (MB) | 5.88 | 0.06 |

#### MobileNetV3-Large

**Method:** One-time structured pruning at a 90% pruning ratio, followed by int8 quantization.

| Metric | Baseline (fp32) | Optimized |
|---|---|---|
| Loss | 0.0829 | 0.4568 |
| Accuracy | 0.9757 | 0.8572 |
| Total unquantized parameters | 4,214,842 | 2,650 |
| Trainable unquantized parameters | 4,214,842 | 722 |
| Size (MB) | 16.17 | 0.11 |

## 3. Conclusion

By pruning and quantizing the model, we were able to maintain acceptable accuracy while drastically reducing its size. For further optimization, we would have conducted more research into iterative pruning, pruning with different hyperparameters, and quantization-aware training (QAT), as these promise better results but would have gone beyond the scope of the course and are left to the reader's interest.

## References


[1] P. Helber, B. Bischke, A. Dengel, and D. Borth, “EuroSAT: A novel dataset and deep learning benchmark for land use and land cover classification,” IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens., vol. 12, no. 7, pp. 2217–2226, Jul. 2019.

[2] A. Howard et al., “Searching for MobileNetV3,” in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), 2019, pp. 1314–1324. Available: https://arxiv.org/abs/1905.02244 [Accessed: Jul. 9, 2026].

[3] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” GitHub repository, 2023. Available: https://github.com/ultralytics/ultralytics [Accessed: Jul. 9, 2026].

[4] R. Liu, “Higher accuracy on vision models with EfficientNet-Lite,” TensorFlow Blog, Mar. 2020. Available: https://blog.tensorflow.org/2020/03/higher-accuracy-on-vision-models-with-efficientnet-lite.html [Accessed: Jul. 6, 2026].

[5] S. Han, J. Pool, J. Tran, and W. J. Dally, “Learning both weights and connections for efficient neural networks,” in Adv. Neural Inf. Process. Syst. (NeurIPS), 2015, pp. 1135–1143.

[6] Y. He and L. Xiao, “Structured pruning for deep convolutional neural networks: A survey,” IEEE Trans. Pattern Anal. Mach. Intell., vol. 46, no. 5, pp. 2900–2919, May 2024.

[7] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, “Pruning filters for efficient ConvNets,” in Proc. Int. Conf. Learn. Represent. (ICLR), 2017.

[8] G. Fang, “Torch-Pruning,” GitHub repository. Available: https://github.com/VainF/Torch-Pruning [Accessed: Jul. 9, 2026].

[9] G. Fang, X. Ma, M. Song, M. B. Mi, and X. Wang, “DepGraph: Towards any structural pruning,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2023, pp. 16091–16101.

[10] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, “A white paper on neural network quantization,” arXiv preprint arXiv:2106.08295, Jun. 2021.

[11] A. Poznanski, “Quantization – PTQ using PyTorch 2 Export Quantization and X86 Backend,” GitHub. Available: https://github.com/arikpoz/neural-network-optimization [Accessed: Jul. 9, 2026].

[12] A. Poznanski, “Quantization – PTQ using PyTorch 2 Export Quantization and XNNPACK Quantizer,” GitHub. Available: https://github.com/arikpoz/neural-network-optimization [Accessed: Jul. 9, 2026].

[13] T. Kim, Y. Yoo, and J. Yang, “FrostNet: Towards quantization-aware network architecture search,” arXiv preprint arXiv:2006.09679, 2020.